

1 Esercizio 1

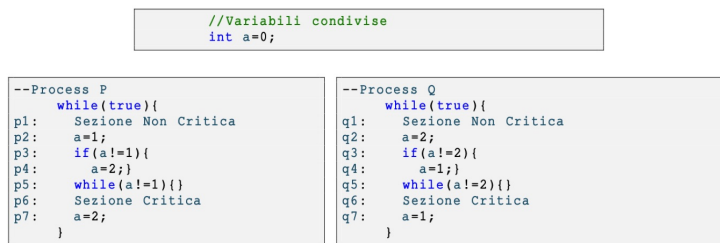
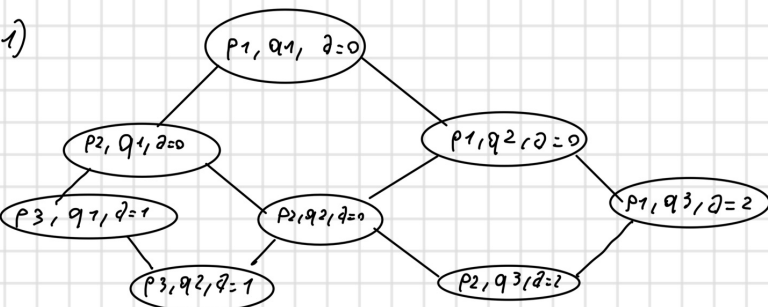


Figura 1: Algoritmo di mutua esclusione

Consideriamo l'algoritmo di mutua esclusione proposto alla figura 1.

1. Disegnare l'inizio dello diagramma degli stati per questo algoritmo (massimo 10 stati).
2. Verifica questo algoritmo la proprietà di mutua esclusione? Giustificare.
3. Verifica questo algoritmo la proprietà di assenza di deadlock? Giustificare.
4. Verifica questo algoritmo la proprietà di assenza di starvation? Giustificare.



2) Non Verifica MUTUA ESCLUSIONE
 Forniamo l'ESCLUSIONE: P1, q1, 0
 P2, //, 0
 P3, //, 1
 P3, //, 1
 P6, //, 1
 // q2, 1
 // q3, 2
 // q5, 2
 // q6, 2

3) SUFFICIENTE CI STA DEADLOCK, QUANTO VUOL DIRE CHE LA VARIABILE CONDIVISA BONA ESISTE CONTEMPORANEAMENTE a=1 E a=2, non E' POSSIBILE

4) NO PERCHÉ BONA ESISTE SEMPRE LO STESSO PROBLEMA

P1, q1, 0
 // q2, 0
 // q3, 2
 // q5, 2
 // q6, 2
 // q7, 2
 // q1, 1
 // q2, 1
 // q3, 2

In uno stadio, ci sono vari giocatori di calcio, alcuni appartengono alla squadra A e altri alla squadra B e vogliono disputare varie partite. Durante una partita si gioca a cinque contro cinque (esattamente), poi, una volta finita, può iniziare una nuova partita. Per iniziare una nuova partita, i giocatori devono aspettare che quelli che hanno partecipato all'ultima partita abbiano finito. Il comportamento di ogni giocatore è il seguente ciclo:

1. Aspetta finché riesce a partecipare ad una partita.
2. Partecipa ad una partita

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, tenendo in considerazione che vogliamo un processo leggero per ogni giocatore (quindi un codice per i giocatori A ed uno per i giocatori B).

3 Esercizio 3

Consideriamo l'algoritmo con message passing proposto sulla figura 2 per 2 processi. Assumiamo che il codice fra [...] si esegue in modo atomico sul sito di ciascun processo.

1. Descrivere in qualche righe come funziona questo algoritmo.
2. Verifica questo algoritmo la proprietà di mutua esclusione? Giustificare.
3. Verifica questo algoritmo l'assenza di deadlock? Giustificare.
4. Verifica questo algoritmo la proprietà di assenza di starvation? Giustificare.

```
--Process Controller
int t=0;
while(true){
c1: }

--Gestione dei messaggi del Controller
case (req,i,t2):
[if(t2==t){
    send(ok)->Pi;
    t=t+1;}
else{
    send(nok)->Pi;
}]
case out:
    t=t+1;
```

```
--Process Pi per i in {0,1}
boolean CS=false;
int t=0;

while(true){
p1: Sezione Non Critica
p2: while(CS==false){
p3:     send(req,i,t)->Controller;}
p4: Sezione Critica
p5:     send(out)->Controller;
p6:     CS=false;
p7:     t=t+2;
}

--Gestione dei messaggi di Pi
--per i in {0,1}
case(nok):
    t=t+2;
case(ok):
    CS=true; -- DA QUI CAPIAMO CHE CS E' UN PERMESSO
```

1) Abbiamo un Controller che gestisce i permessi per il processo P_i per andare in Sezione Critica.

2) Se un processo va in Sezione Critica